

¡De una! Armar una buena documentación es clave para los trabajos prácticos de la facultad. Vamos a armarla con un tono súper claro, directo y humano, explicando qué hace cada archivo y cada bloque de código de una forma que sea fácil de entender (y de defender si te llegan a preguntar los profesores).

Acá tenés la documentación completa del sitio web del **Refugio Patitas Limpias**.

Documentación Técnica: Sitio Web

"Refugio Patitas Limpias"

Materia: Diseño y Desarrollo Web (Trabajo Práctico Nro. 2)

Estructura: Single Page Application (SPA) nativa con HTML5, CSS3 y JavaScript Vanilla.

1. Arquitectura General del Sitio

El sitio está desarrollado bajo el concepto de **SPA (Single Page Application)**. Esto significa que todo el contenido de las distintas páginas solicitadas en las consignas coexiste dentro de un único archivo `index.html`.

Mediante el uso de clases de CSS (`.tab-content` y `.active`) y lógica sencilla en JavaScript, el navegador oculta o muestra las distintas secciones ("páginas") en tiempo real cuando el usuario hace clic en el menú de navegación, evitando que la página tenga que recargarse por completo.

Tanto el **Header (Nav)** como el **Footer** se mantienen siempre visibles y fijos en la pantalla durante toda la navegación.

2. Explicación del Código: HTML (`index.html`)

El archivo HTML contiene el esqueleto y los textos fijos de todas las secciones requeridas.

- **<header> y <nav> (Barra de Navegación):** * Contiene el logo del refugio y una lista (`<ul class="nav-list">`) con botones (`<button>`).
 - Cada botón tiene un atributo especial inventado por nosotros llamado `data-tab` (por ejemplo: `data-tab="inicio"` o `data-tab="contacto"`). Este atributo sirve para que JavaScript sepa exactamente qué sección tiene que mostrar cuando se lo clickea.
 - Incluye un botón con el id `mobile-menu` que contiene el icono de "hamburguesa" (las tres líneas) para desplegar el menú en pantallas chicas.
- **<main> (Contenedor Principal):** * Dentro de este contenedor se agrupan todas las secciones mediante la etiqueta `<section>`. Cada una representa una "página" del sitio y lleva la clase `.tab-content`. La primera sección (`#inicio`) lleva además la clase `.active` para que sea lo primero que se ve al entrar.
- **Sección 1: Inicio (`id="inicio"`)**
 - Tiene un banner principal (`.image_container`) con una foto de fondo y el título de bienvenida.
 - Abajo cuenta con dos tarjetas de contenido (`.card-banner`). La primera cuenta la historia de adopción del perrito **Rocky** y la segunda es una invitación interactiva con formato de banner ("*¡Vení a conocer a los perritos!*") inspirada en la dinámica de refugios reales como *El Campito*, incentivando las visitas de los sábados.
- **Sección 2: Acerca de (`id="nosotros"`)**

- Presenta una descripción humana del propósito del refugio y una grilla (`.about-grid`) dividida en dos bloques esenciales: la **Misión** y la **Visión** de la ONG.
- **Sección 3: ¿Qué hacemos?** (`id="que-hacemos"`)
 - Estructurada en **formato Blog**. Muestra una grilla con tres artículos o proyectos ficticios del refugio (Campañas de castración, Charlas en escuelas y Obras de nuevos caniles). Cada posteo tiene su propia imagen, una etiqueta de categoría, título y bajada de texto.
- **Sección 4: Contacto** (`id="contacto"`)
 - Un formulario (`<form id="contactForm">`) con campos para Nombre, Email, un menú desplegable (`<select>`) para elegir el motivo de contacto, un cuadro de texto para el mensaje y el botón de envío.
- **Sección 5: Landing Page** (`id="donar"`)
 - Es la página de suscripción del refugio. Cuenta con dos tarjetas destacadas con precios ficticios orientadas a que la gente colabore mensualmente (Alimento Mensual y Atención Médica). La tarjeta médica cuenta con un "badge" (etiqueta pequeña) que indica que es la opción más urgente.

3. Explicación del Código: CSS (`styles.css`)

El archivo de estilos se encarga de que el sitio se vea ordenado, moderno y, lo más importante, que cumpla con las reglas de adaptabilidad.

- **Reset General (*)**: Setea los márgenes y paddings en `0` y activa `box-sizing: border-box` para que el cálculo de los tamaños de las cajas sea exacto y no se rompa el diseño.
- **Layout base (Escritorio)**: Usa propiedades de **Flexbox** (`display: flex`) en la barra de navegación, en las tarjetas de inicio, en la sección de Misión/Visión y en el Blog. Esto permite alinear los elementos de forma horizontal muy fácilmente.
- **El truco del intercalado** (`.card-banner:nth-child(even)`): Esta regla busca las tarjetas que sean pares (la segunda, la cuarta, etc.) y les aplica la propiedad `flex-direction: row-reverse`. De esta forma, la primera tarjeta muestra la foto a la izquierda y el texto a la derecha, y la segunda tarjeta invierte el orden automáticamente (foto a la derecha y texto a la izquierda), logrando un diseño súper dinámico sin tocar el HTML.
- **Ocultar/Mostrar pestañas**: * `.tab-content` tiene por defecto un `display: none` (lo que hace que todas las secciones estén ocultas).
 - `.tab-content.active` tiene un `display: block`, lo que significa que solo la sección que tenga la clase `.active` se va a mostrar en pantalla.

Explicación de los Breakpoints (Diseño Responsive)

Para cumplir con las consignas estrictas del trabajo práctico, se dividió el diseño adaptivo en los dos rangos exactos solicitados:

1. **Rango Celulares** (`@media (max-width: 600px)`):
 - El menú de navegación horizontal se oculta (`display: none`) y se transforma en un menú vertical absoluto que se despliega sobre el contenido al tocar el botón de tres líneas.
 - Las grillas horizontales (las tarjetas de inicio, las cajas de misión/visión, los posts del blog y las opciones de donación) pasan a tener `flex-direction: column`. Esto hace que todo el contenido fluya verticalmente hacia abajo,

ideal para leer cómodamente desde una pantalla táctil sin que nada se corte o se deforme.

2. Rango Tablets (@media (min-width: 600px) and (max-width: 992px)):

- Aquí el menú vuelve a ser horizontal pero con fuentes y espaciados más compactos para cuidar el espacio.
- Las tarjetas de inicio se achican proporcionalmente.
- En la sección de Blog (.blog-grid), los posts se configuran para ocupar el 50% del contenedor menos el espacio de separación (flex: 0 0 calc(50% - 10px)). Esto permite mostrar un diseño de **2 columnas** ordenadas, ideal para el ancho de pantalla de una tablet.

4. Explicación del Código: JavaScript (script.js)

El script está escrito con funciones y variables tradicionales (var), priorizando un código claro, limpio y directo.

- **Menu Mobile (Hamburguesa):**
 - Escucha cuando se hace clic en el botón #mobile-menu y le agrega o le quita la clase open a la lista de navegación (navList.classList.toggle('open')). El CSS se encarga de mostrarla cuando esa clase está activa.
- **Lógica de Navegación SPA:**
 - Selecciona todos los botones del menú (.nav-link) y les añade un evento de escucha (addEventListener('click')).
 - Al hacer clic en un botón, lee su atributo data-tab para saber a qué sección quiere ir el usuario.
 - Primero, le remueve la clase active a todos los botones y a todas las secciones para "limpiar" la pantalla.
 - Segundo, le clava la clase active únicamente al botón apretado y a la sección que coincide con el ID que guardamos en data-tab.
 - Finalmente, si el usuario está en el celular, cierra el menú desplegable automáticamente y hace un scroll suave hacia el tope de la pantalla (window.scrollTo) para que la nueva sección empiece desde arriba.
- **Manejo del Formulario de Contacto:**
 - Captura el evento submit del formulario. Usamos event.preventDefault() para frenar el comportamiento nativo del navegador (que recargaría la página y rompería la SPA).
 - Guarda los valores de los inputs en variables (nombre, email, motivo), lanza un alert() amigable confirmando que los datos se enviaron bien y limpia los campos de texto con formulario.reset().
- **Funciones Auxiliares:**
 - donarPlan(nombrePlan): Dispara una alerta interactiva avisando qué plan de suscripción mensual seleccionó el usuario en la Landing Page.
 - irAContacto(): Es la función que usan los botones de las tarjetas de inicio ("Enterate cómo" y "¿Cómo venir?"). Muestra un aviso al usuario y simula un clic real en el botón de "Contacto" del menú (botonContacto.click()), aprovechando la lógica del SPA para redirigir al usuario al formulario de forma automática.

Como agregar una pagina desde JS

Básicamente consiste en 3 pasos simples:

1. **En el HTML:** Metés **todas** tus páginas dentro del mismo archivo, metidas cada una adentro de un `<section>`. A cada sección le ponés un **id** único (que sería el nombre de tu página) y una clase común (por ejemplo, `tab-content`).
2. **En el CSS:** Hacés que por defecto todas las secciones con la clase `tab-content` estén ocultas (`display: none`). Y creás una clase especial (por ejemplo, `.active`) que tenga `display: block` para mostrar únicamente la que tenga esa clase.
3. **En el JS:** Creás una función que "escuche" cuando alguien hace click en el menú. Cuando hacen click, JS le saca la clase `.active` a la página que se estaba viendo y se la pone a la nueva sección que el usuario quiere abrir.

Guía paso a paso para agregar una página nueva

Imaginate que ahora querés agregar una página nueva llamada **"Voluntarios"**. Estos son los 3 retoques que tendrías que hacer en tus archivos:

Paso 1: Modificar el HTML (`index.html`)

Primero, agregás el botón en tu menú de navegación (`<nav>`) con un atributo `data-tab` que inventamos nosotros para indicarle el nombre de la sección. Después, creás la sección abajo de todo dentro del `<main>`:

HTML

```
ul class "nav-list" id "nav-list"
  li button class "nav-link active" data-tab "inicio" button li
  li button class "nav-link" data-tab "contacto" button li
  li button class "nav-link" data-tab "voluntarios" button li
ul

main
  section id "inicio" class "tab-content active" section
  section id "contacto" class "tab-content" section

  section id "voluntarios" class "tab-content"
    div class "about-container"
      h2
      p
    div
  section
main
```

Paso 2: El comportamiento en el CSS (`styles.css`)

Para el CSS no tenés que agregar nada nuevo por cada página, porque la estructura ya es genérica. La magia la hacen estas dos reglas que ya tenés armadas en tu archivo:

CSS

```
/* Esto hace que todas las páginas nazcan ocultas e invisibles */
.tab-content
```

```
/* Cuando JavaScript le pega la clase 'active' a una sección, la muestra en pantalla */
.tab-content.active
```

Paso 3: El recorrido en el JavaScript (script.js)

Tu código de JavaScript ya está preparado de forma "inteligente" para leer de forma automática cualquier botón nuevo que agregues, gracias a que usa un bucle (forEach).

Mirá lo que hace el código que armamos línea por línea:

JavaScript

```
// 1. Agarramos todos los botones que tengan la clase '.nav-link'
var document '.nav-link'

// 2. Agarramos todas las secciones de contenido que tengan la clase '.tab-content'
var document '.tab-content'

// 3. Con un forEach, le decimos a JS que vigile a TODOS los botones del menú
function
  'click' function

// 4. Guardamos en una variable el nombre de la pestaña que quiere abrir el usuario
// Si clickeó el botón nuevo, 'targetTab' va a valer "voluntarios"
var 'data-tab'

// 5. Limpieza: Le sacamos la clase 'active' a todos los botones para que no queden todos
pintados
function
  'active'

// Le ponemos la clase 'active' solo al botón que se acaba de clicar
  'active'

// 6. El cambio de página: Ocultamos todas las secciones del sitio
function
  'active'

// ¡Acá está la magia! Si el ID de la sección coincide con el 'data-tab' del botón...
if
  // ... le clavamos la clase 'active' para que el CSS la muestre en la pantalla
    'active'

// 7. Si estamos en el celu y el menú desplegable estaba abierto, lo cerramos
if
  'open'
  'open'

// 8. Llevamos el scroll arriba de todo de forma suave para arrancar a leer la página desde
el principio
window top 0 behavior 'smooth'
```

Resumen para cuando quieras agregar otra:

Cada vez que el profe te pida una página nueva o quieras inventar otra:

1. Agregás un `<li><button class="nav-link" data-tab="nombre-de-tu-pagina">Texto</button></li>` en el menú.
2. Agregás un `<section id="nombre-de-tu-pagina" class="tab-content"> ... </section>` adentro del `<main>`.
3. ¡Listo! No hace falta que toques ni el CSS ni el JS porque el código ya reconoce automáticamente los nuevos elementos por las clases.